



Installation of AMSTer

(FOR Mac OS DISTROS)

Installation guide by:

*N. d'Oreye, D. Smittarello, M. Jaspard, G. Celli, Q. Glaude – ECGS
B. Smets – RMCA*

https://github.com/AMSTerUsers/AMSTer_Distribution

Version 7.0: Aug 20 2026

Preliminary information.....	2
0) Prepare the work.....	5
1) Installing some external components.....	6
2) Installing required tools	10
3) Installing AMSTer software	18
4) Directory structure for data, auxiliary data and results.....	24
5) Check installation	25
6) Operating with cron jobs	29
7) Downloading the Sentinel-1 orbits	30
8) Updates in this manual.....	31

Preliminary information

AMSTer (Automated SAR & InSAR Mass processing Software for Multidimensional Time series) was initially named **MasTer**. Some publications refer to that former name.

AMSTer is composed of 3 main elements: **AMSTer toolbox**, **AMSTer Engine** and **MSBAS**.

AMSTer toolbox is a set of mostly shell scripts aiming at processing automatically a large number of interferometric pairs and feeding and running the **MSBAS** processor [Samsonov and d'Oreye, 2012, 2017; Samsonov et al., 2017, 2020] in order to obtain the desired deformation maps and time series in vertical and horizontal components. Geocoded amplitude, coherence, interferometric phase and deformation maps are computed using the **AMSTer Engine**, a command line InSAR processor derived from the Centre Spatial de Liege (CSL) InSAR Suite (CIS) [Derauw, 1999; Derauw et al, 2019].

This guide explains the installation procedure for **AMSTer (AMSTer toolbox, AMSTer Engine, MSBAS)**, on a Mac computer. It does not replace the [manual](#) file provided with the software distribution ([AMSTer_Manual_x.x.docx](#)), but rather provide much more details to allow any user with less-advanced knowledge of the Mac environment to properly install the software suite.

The installation procedure has been last tested with the **AMSTerEngine_v20260813** package, msbasv3 (20190121), msbasv4 (20201009), msbasv10 (20230601) and shell scripts dated from August 2026.

Sources and codes are licensed under GNU Affero General Public (AGPL) License¹ and freely available for noncommercial use from Github:

https://github.com/AMSTerUsers/AMSTer_Distribution

If you are not granted the access to the repository or the discussions group, please send an email to ndo@ecgs.lu.

Download:

Either from the **distribution repository**:

- Only the installer script (**[AMSTer_install.sh](#)**) if you do not need/want to make a manual installation at https://github.com/AMSTerUsers/AMSTer_Distribution/tree/main/Installer (**recommended**), or
- the full package at https://github.com/AMSTerUsers/AMSTer_Distribution

Or from the **development (private) repositories**:

- the documentation and manuals from [ndoreye/DOCS](#)
- the scripts from [ndoreye/SCRIPTS_MT](#)
- the MSBAS sources tuned for AMSTer Toolbox from [ndoreye/MSBAS](#) (also available directly from S. Samsonov; see manual)
- the AMSTerEngine sources from [ndoreye/_Sources_AE](#) (also available directly from D. Derauw; see manual)

Note: Gitkraken is a convenient interface for Github but is not mandatory.

¹ as published by the Free Software Foundation, either version 3 of the License, or any later version. See the GNU Affero General Public License at <https://www.gnu.org/licenses/>.

This installation procedure has been tested on:

- **MacOS Monterey version 12.7, Sonoma 14.7 and Tahoe 26.6.1**

though former installations were operational already on Mojave (V10.14.6) and former MacOS versions.

It is assumed that the user has the administrator rights on the computer and can provide the root privilege when the “sudo” command is executed in the terminal.

In this installation guide, each command line that has to be written and executed in a terminal is written in *italic blue*. Files names are in *blue*. Paths are given in *green*.

Note:

A script exists to assist you making the installation. See

.../SAR/AMSTer/SCRIPTS_MT/Installer/AMSTer_install.sh

(in distribution repository), or

.../SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT/AMSTer_install.sh

(in development repository).

It is highly recommended to use its version 7.0 or above, which install python3.11 in the virtual environment as described here and download the sources for you from GitHub.

Read carefully all the text and questions displayed by the script.

For older Mac OS versions, you may need to adapt the script or proceed manually by adapting the procedure below.

Note that the ***AMSTer_install.sh*** installation script may be run several times (either to correct a problem during the installation or to perform an update of any element at any time). However, if some elements were installed manually (eg Gitkraken, Fiji or QGIS...), the script may not properly detect them as they may have been installed in an expected. This must not be a problem though.

To make the installation using the *AMSTer_install.sh* script (highly recommended**), refer to then user manual (chapter 0.5).**

The script will make for you all the installations described in sections 0 - 3 here after.

Kind reminder: The software is provided for free. To help us carrying on with the development of the tool, we kindly ask you to cite the **AMSTer Software** in publications that contains results produced by the toolbox thanks to at least the most recent references from below:

For AMSTer

- Derauw, D., N. d'Oreye, M. Jaspard, A. Caselli, and S. Samsonov (2020), Ongoing automated ground deformation monitoring of Domuyo - Laguna del Maule area (Argentina) using Sentinel-1 MSBAS time series: Methodology description and first observations for the period 2015-2020., *Journal of South American Earth Sciences*, 104, 102850, doi:10.1016/j.jsames.2020.102850.
- d'Oreye N., Derauw, D., S. Samsonov, M. Jaspard, and D. Smittarello (2021), MasTer: a full automatic multi-satellite InSAR mass processing tool for rapid incremental 2D ground deformation time series. *Proc. IEEE IGARSS21*, Brussels, July 2021.
- Smittarello, D., d'Oreye, N., Jaspard, M., Derauw, D., & Samsonov, S. (2022). Pair selection optimization for InSAR time series processing. *Journal of Geophysical Research: Solid Earth*, 127, e2021JB022825.
- d'Oreye N., Derauw, D., S. Samsonov, M. Jaspard, D. Smittarello, G. Celli (2023), AMSTer: SAR & InSAR Automated Mass processing Software for Multidimensional time series. *Instruction manual V6.0*, Nov. 2023. 237 pp.

For AMSTer Engine

- Derauw, D., (1999). Phasimétrie par Radar à Synthèse d'Ouverture; théorie et applications. PhD Dissertation, University of Liège, 141 pages.
- Libert, L., Derauw, D., d'Oreye, N., Barbier, C., Orban, A., (2017). Split-band interferometry-assisted phase unwrapping for the phase ambiguities correction. *Remote Sensing*, 9(9), 879.

For MSBAS

- Samsonov, S., d'Oreye, N., (2012). Multidimensional time-series analysis of ground deformation from multiple InSAR data sets applied to Virunga Volcanic Province. *Geophysical Journal International* 191(3), 1095–1108. doi: 10.1111/j.1365-246X.2012.05669.x.
- Samsonov, S., d'Oreye, N., (2017). Multidimensional Small Baseline Subset (MSBAS) for two-dimensional deformation analysis: Case study Mexico City. *Canadian Journal of Remote Sensing* 82(6), 1–12. doi: 10.1080/07038992.2017.1344926
- Samsonov S., W. Feng, A. Peltier, H. Geirsson, N. d'Oreye, KK. F.Tiampo (2017). Multidimensional Small Baseline Subset (MSBAS) for volcano monitoring in two dimensions: opportunities and challenges. Case study Piton de la Fournaise volcano. *Journal of Volcanology and Geothermal Research*, Vol 344, 121-138, <https://doi.org/10.1016/j.jvolgeores.2017.04.017>
- Samsonov, S., A. Dille, O. Dewitte, F. Kervyn, and N. d'Oreye (2020). Satellite interferometry for mapping surface deformation time series in one, two and three dimensions: A new method illustrated on a slow-moving landslide. *Engineering Geology*, (266), 105471, doi:10.1016/j.enggeo.2019.105471.items

For DORIS

- Kampes, B.M., Hanssen, R.F., Perski, Z., (2003). Radar interferometry with public domain tools. In: *Third International Workshop on ERS SAR Interferometry, FRINGE03*, Frascati, Italy, 1-5 Dec. 2003, 6 pages.

0) Prepare the work

The tools will need the following directories:

- /opt/local/bin
- \$HOME/SAR/EXEC
- \$HOME/SAR/AMSTer/DOC
- \$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older
- \$HOME/SAR/AMSTer/MSBAS
- \$HOME/SAR/AMSTer/SCRIPTS_MT

Where \$HOME/ is obviously your home directory.

It is also advised to source your **.barshrc** in your **.bash_profile** as all the config will be set in **.bashrc**.

If you do not have these files, create them and ensure you can write and execute them.

If you have old AMSTer installation parameters in your **.bashrc**, it is advised to remove them and write new ones based on the instructions below.

1) Installing some external components

Xcode:

- Check first whether the Xcode Command Line Tools are already installed. This is tested with:
`pkgutil --pkg-info=com.apple.pkg.CLTools_Executables`
- If the command answers nothing, install them by typing at the terminal:
`xcode-select --install`
- **Beware: on a Mac, `/usr/bin/make` and `/usr/bin/git` are not the real tools. They are small shims that ask xcode-select where the real ones live. They exist even on a brand new Mac with no developer tool at all, and they fail after a major OS upgrade (e.g. to Tahoe) or when Xcode.app was removed while its path stayed selected, although pkgutil above still reports the Command Line Tools as installed. Check the ACTIVE toolchain before compiling anything:**
`xcode-select -p`
`ls "$(xcode-select -p)"/usr/bin/make`
- If make is not there, re-select the Command Line Tools (or the full Xcode if you have one):
`sudo xcode-select -s /Library/Developer/CommandLineTools`
`sudo xcode-select -s /Applications/Xcode.app/Contents/Developer`
- If it remains broken, force a clean reinstall of the Command Line Tools, then start again:
`sudo rm -rf /Library/Developer/CommandLineTools`
`sudo xcode-select --install`
- As a last resort, the GNU make (gmake) installed by Homebrew or MacPorts does not depend on xcode-select at all and can be used instead of make in every compilation below.

AMSTer uses MacPorts on macOS before 26 (Tahoe), and Homebrew from macOS 26, because some of the options and variants needed here are not installable through MacPorts anymore. One need to test the major version returned by typing

o `sw_vers -productVersion`

and sets its `PKGMR` variable to port or brew accordingly; everything below follows that choice. Do not mix both package managers for the AMSTer dependencies.

Macports (for macOS < 26, i.e. before Tahoe):

- Download latest version from www.macports.org > Available Downloads:
 - o E.g., for Monterey, take [MacPorts-2.7.2-12-Monterey.pkg](#)
 - o Open and install it.
It can also be installed from the Terminal, once the package has been stored in `$HOME/SAR/EXEC`:
 - `sudo installer -pkg $HOME/SAR/EXEC/MacPorts-x.y.z-YourOS-Name.pkg -target /`
 - o Check that it worked before going any further; if not, install MacPorts manually:
 - `command -v port` => it must answer `/opt/local/bin/port`
 - o And/or update the ports (advised at each installation or update; it may take a long time and requires your admin password):
 - `sudo port selfupdate`
 - `sudo port upgrade outdated`
 - `sudo port reclaim`

- If your OS is 10.15 or more recent, it is required to change the default shell Zsh with bash for scripts compatibility issues. If needed, runs:
 - `chsh -s /bin/bash`

The change is only effective in a new Terminal, hence close the present Terminal and carry on with the installation in a new one.

Homebrew (for macOS 26 Tahoe and later):

- Detect the real hardware architecture of your Mac, even if this Terminal happens to run translated under Rosetta 2:
 - `uname -m` => answers `arm64` or `x86_64`
- If it answers `x86_64`, check whether this shell is translated:
 - `sysctl -n sysctl.proc_translated`
 - => if it returns 0: this Mac really is `x86_64` (Intel)
 - => if it returns 1: the hardware is `arm64` (Apple Silicon) and this shell is translated. Do NOT install anything from there: mixing translated and native tools causes real and hard to diagnose problems. Relaunch a native shell first:
 - `arch -arm64 /bin/bash`
- Get Homebrew (if not there yet).
 - `sudo -v`
 - `NONINTERACTIVE=1 arch -"YourArchitecture" /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`

Where *YourArchitecture* is either `arm64` or `x86_64`.
- Homebrew installs itself in `/opt/homebrew` on Apple Silicon and in `/usr/local` on Intel. If only the NON-native one is found (e.g. a `/usr/local` Homebrew on an Apple Silicon Mac), it still works but it is slower and it can cause architecture mismatches when compiling AMSTerEngine or msbas: install the native one as well. Make brew usable in the present shell (and add the same line in your `.bashrc`) with:
 - `eval "$(/opt/homebrew/bin/brew shellenv)"`
- Make it clean and up to date
 - `brew update`
 - `brew upgrade`
 - `brew cleanup`

GMT and GDAL:

GMT, GDAL and some associated utilities are required for plots, images and GIS manipulations.

- Using MacPorts (`macOS < 26`), at Terminal, type:
 - `sudo port clean gdal`
 - `sudo port install openjpeg`
 - `sudo port install gdal gdal-netcdf libkml postgresql17`
 - `sudo port install gmt6`
 - `sudo port install GraphicsMagick`
 - `sudo port install ffmpeg`

Note: if GDAL is crashing (e.g. after updates when most recent version are not 100% compatible yet), you may try installing `homebrew` version. This however may require recompiling msbas and/or AMSTer Engine sources after an OS update as homebrew does not manage libraries the same way as MacPorts.

- Using Homebrew (**macOS 26 Tahoe and later**), the gdal formula is already built with netcdf, hdf5 and openjpeg support, hence there is no variant to add:
 - o `brew install openjpeg`
 - o `brew install gdal`
 - o `brew install postgresql@17`
 - o `brew install gmt`
 - o `brew install graphicsmagick`
 - o `brew install ffmpeg`

In both cases, the binary is named gmt6 with MacPorts and gmt with Homebrew. Link it to the same `/opt/local/bin/gmt` so that the scripts are portable:

- o `sudo mkdir -p /opt/local/bin`
- o `sudo ln -sf "$(which gmt || which gmt6)" /opt/local/bin/gmt`

and check the installed version with:

- o `gdalinfo --version`

The following tools and apps are not mandatory, but experience shows that it may greatly help the user.

GitKraken :

Although not mandatory, Gitkraken is a useful tool to sync with the last versions of AMSTer Toolbox.

- Download [GitKraken](https://www.gitkraken.com) (from <https://www.gitkraken.com>) and connect e.g. via [Github](#)
- Create profile in GitKraken
- Launch GitKraken
- Clone Repo > with URL :
 - o Where to Clone: `$HOME/SAR/`
 - o Repo to clone: URL to AMSTerSoftware_Distribution

To display raster figure: XnView, GIMP, nomacs or Graphic Converter

- my favourite is [Graphic Converter](#) though it is not free...
- A very good alternative is [nomacs](#):
 - > visit the web site (<https://nomacs.org/docs/getting-started/installation/>) and install manually for your hardware.
- To install GIMP:
 - o `sudo port install "gimp2"` (For Mac OSX < 26)
 - o `brew install --cask gimp2` (For Mac OSC 26 or later)

A text editor: BBEdit (not free though)

- Get it from <https://www.barebones.com/products/bbedit/download.html> or any other convenient editor that you like

Cyberduck

- Get it from <https://cyberduck.io>

or any other convenient ftp software that you like

gnu fortran (not mandatory, and no longer installed by the installer script):

- Get it from <https://github.com/fxcoudert/gfortran-for-macOS/releases>
- install

To display geocoded products: QGIS

- Download latest version from <https://qgis.org/en/site/forusers/download.html>
- note some interesting plugins are:
 - o [point sampling tool](#)
 - o [PointConnetor](#)
 - o [Profile tool](#)
 - o [Qdraw](#)
 - o [QuickMapServices](#)
 - o [RasterDataPlotting](#) (may require to install python first)
 - o [Serval](#)
 - o [Temporal/Spectral Profile Tool](#)
 - o [Value Tool](#)

To quick look scripts with colours when hitting the space bar: QLColorCode

- o *[sudo port install QLColorCode](#)*
- o *[reboot](#)*

2) Installing required tools

AMSTer needs some pre-existing software and libraries to work properly. This is what we call the “dependencies”. So, we first need to install them. These installations are explained here, sorted by type of installation procedure.

bashrc:

- Open Terminal
- Type: `touch .bashrc`
- Type: `nano .bashrc`
- Copy/paste infos from [example_bashrc.txt](#) eg using nano editor
- Adapt to needs (see below)
- Exit nano by Crtl-X > Y
- Create a `.bash_profile` that will use your `.bashrc`. Type: `touch .bash_profile`
- Type: `nano .bash_profile`
- Copy/paste the following lines :


```
if [ -r ~/.bashrc ] ; then
    source ~/.bashrc
fi
```
- Exit nano by Crtl-X > Y
- Type:


```
chmod 700 .bashrc
chmod 700 .bash_profile
```
- Reboot

Note: for recent Mac, default shell is not bash. See here how to get it back to default bash: <https://www.howtogeek.com/444596/how-to-change-the-default-shell-to-bash-in-macos-catalina/>

Libraries & modules:

Install libraries and modules using MacPorts (< **macOS 26**) or Homebrew (**macOS 26 Tahoe and later**). The Homebrew names differ for a few of them.

Using MacPorts:

- C/C++ compiler.

Version 20 or above is mandatory from AMSTerEngine_20250805. With MacPorts, several versions are installed because different components require different ones:

 - o `sudo port install clang-14` (the only usable one on macOS 10.15 and older)
 - o `sudo port install clang-20`
 - o `sudo port install clang-21` (used by the msbas makefile)
- others
 - o `sudo port install tiff`
 - o `sudo port install libgeotiff`
 - o `sudo port install fftw-3-long`
 - o `sudo port install fftw-3-single`
 - o `sudo port install hdf5`

Note: At compilation, a note is displayed reading:
Mac users may need to set the environment variable "HDF5_USE_FILE_LOCKING" to the five-character string "FALSE" when accessing network mounted files. This is an application run-time setting, not a configure or build setting. Otherwise errors such as "unable to open file" or "HDF5 error" may be encountered.
Not sure what to do out of it yet though...
 - o `sudo port install ImageMagick`
 - o `sudo port install libxml2`

- `sudo port install lapack`
- `sudo port install libomp` (mandatory for the parallelised compilation of AMSTerEngine and msbas; replaces the former libomp-devel)
- `sudo port install ghostscript` (used by GraphicsMagick for EPS/PS/PDF)
- `sudo port install mpich` (required by msbasv10)
- `sudo port install parallel`
- `sudo port install gsl`

Using Homebrew

- With Homebrew, the numbered clang releases are not kept as MacPorts does; only the latest llvm/clang is available, which is well above the required v20:
 - `brew install llvm`

Note: a Makefile that expects a MacPorts style binary name such as clang++-mp-21 will not find it under Homebrew. Point it to `$(brew --prefix llvm)/bin/clang` instead, or use the Homebrew flavoured Makefile when the sources provide one (see 3.2).
- others
 - `brew install libtiff`
 - `brew install libgeotiff`
 - `brew install fftw`²
 - `brew install imagemagick`
 - `brew install hdf5`
 - `brew install libxml2`
 - `brew install lapack`
 - `brew install libomp` (mandatory for the parallelised compilation of AMSTerEngine and msbas; replaces the former libomp-devel)
 - `brew install ghostscript` (used by GraphicsMagick for EPS/PS/PDF)
 - `brew install mpich` (required by msbasv10)
 - `brew install parallel`
 - `brew install gsl`

Locate the ImageMagick convert command, because the directory that contains it must be declared as `PATHCONV` in your `.bashrc`:

- `which convert`

Note: Recent ImageMagick 7 formulae may no longer provide the **convert** compatibility shim, in which case the command is named **magick**. And with Homebrew, a leftover MacPorts ImageMagick would be found first since `/opt/local/bin` is deliberately early in the PATH. To avoid both problems, let's mirror the right binary in `/opt/local/bin` and sets `PATHCONV` to that single location:

- `sudo ln -sf "$(brew --prefix imagemagick)/bin/magick" /opt/local/bin/convert`
- `export PATHCONV=/opt/local/bin`

² for both fftw-3-long and fftw-3-single (the single formula builds single, double and long double)

GNU functions and utilities:

Mac version of some tools such as [sed](#) or [awk](#) etc do not have exactly the same syntax as their gnu version. For the sake of portability (and because some options are not available for Mac versions), we use here gnu tools:

Using MacPorts:

- At Terminal:

- o [sudo port install gsed](#)
- o [sudo port install gawk](#)
- o **Note: At compilation, a note is displayed reading:**
 readline support has been removed from gawk. If you need to run gawk interactively, install rlwrap:
 [sudo port install rlwrap](#)
 and run gawk using rlwrap:
 [rlwrap gawk ...](#)
- o [sudo port install coreutils](#) (i.e. for gdate, gstat)
- o [sudo port install findutils](#) (i.e. for find)
- o [sudo port install grep](#) (i.e. for ggrep)
- o [sudo port install wget](#) (needed i.e. to download the S1 orbits)
- o [sudo port install curl](#)

Using Homebrew:

- At Terminal:

- o [brew install gnu-sed](#)
- o [brew install gawk](#)
- o [brew install coreutils](#) (i.e. for gdate, gstat)
- o [brew install findutils](#) (i.e. for find)
- o [brew install grep](#) (i.e. for ggrep)
- o [brew install wget](#) (needed i.e. to download the S1 orbits)
- o [brew install curl](#)

Check that all these gnu functions are stored in the [/opt/local/bin](#) directory. If not, move them there. That directory **MUST** be defined as `${PATHGNU}` state variable in your [.bashrc](#). At Terminal, type

- o [which gsed](#)
- o [which gawk](#)
- o [which gdate](#)
- o [which gstat](#)
- o [which gfind](#)
- o [which ggrep](#)

If answer to all these lines is not the same as `echo ${PATHGNU}`, edit your [.bashrc](#) and change `${PATHGNU}` state variable. It must be something like
[export PATHGNU=/opt/local/bin](#)

Don't forget that a reboot (or source [.bashrc](#)) is required for changes in [.bashrc](#) to be taken into account.

Note: ensure that [/opt/local/bin](#) is in your path before the [/usr/bin](#) to ensure that it would in priority take into account your GNU commands in case of doubt.

Just in case one of these functions would be called without its g-name in a script, it is recommended to have them in `${PATHGNU}` with both names, that is with and without heading g (that is **sed** and **gsed**, **awk** and **gawk**, **grep** and **ggrep**, **date** and **gdate**, **stat** and **gstat**. This can be simply done with a link.

- In Terminal, type:

- o [cd \\${PATHGNU}](#)
- o [ln -s gsed sed](#)
- o [ln -s gawk awk](#)

- `ln -s gdate date`
- `ln -s gstat stat`
- `ln -s gfind find`

Note that `{PATHGNU}` stays `/opt/local/bin` in all cases, including with Homebrew whose gnu tools really live in `/opt/homebrew/bin` or `/usr/local/bin`. This is on purpose, so that `PATHGNU` is identical on Mac (MacPorts or Homebrew) and on Linux and every script hardcoding `"${PATHGNU}/gsed"` keeps working.

With Homebrew, do not copy the links below but create them from the real binaries, e.g.:

```
sudo ln -sf "$(which gsed)" /opt/local/bin/gsed and
sudo ln -sf "$(which gsed)" /opt/local/bin/sed
```

And to be sure...(I know it is overkill, but one never know...):

- `ln -s ggrep grep`
- `ln -s gseq seq`
- `ln -s guniq uniq`
- `ln -s greadlink readlink`
- `ln -s gdu du`
- `ln -s gxargs xargs`

Check also that `wget` is in `{PATHGNU}` with `which wget`. If it is not, link it there:

- `sudo ln -sf "$(which wget)" {PATHGNU}/wget`

Check as well where `curl` is installed with `which curl`, because that directory must be added to your `PATH` in the `.bashrc`.

For some plotting applications: Java

With Homebrew, simply install the latest `openjdk`:

- `brew install openjdk`

Homebrew's `openjdk` is keg-only, i.e. it never puts itself on the `PATH` and the system does not see it. Register it and make sure it is not shadowed by a leftover MacPorts java (remember that `/opt/local/bin` comes early in the `PATH`):

- `sudo mkdir -p /Library/Java/JavaVirtualMachines`
- `sudo ln -sf "$(brew --prefix openjdk)/libexec/openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk.jdk`
- `sudo ln -sf "$(brew --prefix openjdk)/bin/java /opt/local/bin/java`
- `sudo ln -sf "$(brew --prefix openjdk)/bin/javac /opt/local/bin/javac`

With MacPorts, the `jdk` is provided as `openjdkNN` ports. The installer script picks the most recent one (`NN`) that can be installed without a full Xcode installation. List the available versions with:

- `port search openjdk | gawk '/^openjdk[0-9]+/{print $1}' | sort -V`

then test them from the most recent one down; a version is unusable if the answer of the dry run below mentions a full Xcode installation:

- `sudo port -n install openjdkNN`

and install the first usable one:

- `sudo port install openjdkNN`

or manually:

- see <http://www.java.com>
- follow instructions to install Java

Notes:

- if some scripts crash with the following message, re-install java and then re-install Fiji:
The operation couldn't be completed. Unable to locate a Java Runtime that supports (null).
- One may need to define `JAVA_HOME` state variable in the `.bashrc` for some application using it. Check where it is installed by typing:
 - o `java -XshowSettings:properties -version 2>&1 > /dev/null | grep 'java.home' | cut -d = -f2- | cut -d " " -f2-`
 and update the `.bashrc` by adding a line with the answer of the line above:
 - o `export JAVA_HOME=PATH_FROM_LINE_ABOVE`
- With Homebrew, do not rely on the bare java command here for the same keg-only reason: `JAVA_HOME` must be set to
`$(brew --prefix openjdk)/libexec/openjdk.jdk/Contents/Home`

Fiji/ImageJ:

- Download latest version from <https://imagej.net/software/fiji/downloads>
- Install
- In Finder, locate `Fiji.app` in your `/Applications` directory. Right click on it and click on "Show Package Contents"
- Navigate in Contents>MacOS and copy the path to the executable. Fiji changed its launcher in February 2025, hence two cases are possible:
 - o **before February 2025**: the executable is
`/Applications/Fiji.app/Contents/MacOS/ImageJ-macosx`
 - o **from February 2025** (Jaunch launcher): the executable is simply named `fiji`, in
`/Applications/Fiji`
- The installer script searches for both, in that order. Find yours with:
 - o `find /Applications/Fiji*/Contents/MacOS/ -type f -name "ImageJ-macosx*" 2>/dev/null`
 - o `find /Applications/Fiji -type f -name "fiji" 2>/dev/null`
- Update `$PATHFIJI` in your `.bashrc` with the directory that contains the launcher found above. It must be something like
 - o `export PATHFIJI=/Applications/Fiji.app/Contents/MacOS/` (before Feb 2025),
or
 - o `export PATHFIJI=/Applications/Fiji` (from Feb 2025).

snaphu:

- Download latest version (e.g. `snaphu-v2.0.6.tar`) from <https://web.stanford.edu/group/ra-dar/softwareandlinks/sw/snaphu/> and store it in in `$HOME/SAR/EXEC`.
- unzip it e.g. using `tar -zxvf`
- edit the Makefile in the `/src` directory to specify your compiler and the optimization flags.
 The default CFLAGS line of snaphu (since v2.0.7) forces `-arch x86_64`, which is not compatible with every architecture, and it must have no static option.
 The installer script rewrites that line with:
`gsed -i 's/^CFLAGS.*CFLAGS = $(OPTIMFLAGS) -Wall/' Makefile`
- compile: at Terminal in `/src` dir, type `make`
 (it seems that you can ignore warnings; most recent compilers are more sensitive)
- Move compiled `snaphu` from `/bin` into `$HOME/SAR/EXEC`
- Download also the `snaphu_man1.txt` if you need more info about snaphu and its configurations.

- Store the installed source e.g. in `$HOME/SAR/EXEC/Sources_Installed`.

cpxfiddle:

Not mandatory, but pretty convenient for creating raster files for quick looking at results.

- Download source from https://github.com/TUDELFTGeodesy/Doris/tree/master/sar_tools
- Cut/paste source in new document with your favorite editor
- Save the file in `$HOME/SAR/EXEC/cpxfiddle.cc`
- In Terminal:
 - o `cd $HOME/SAR/EXEC`
- From macOS 15 (and on Linux), the source must first be patched, otherwise the compilation fails. Replace the line starting with `if (argv[optind]==` by:
 - o `if (argv[optind]==0 || argv[optind][0]!='0')`
- If the compilation still fails because of the register keyword (just a compiler hint, ignored by real compilers for decades), delete that keyword in the three spots where it appears (lines 256, 257 and 635).
 - o `make -n cpxfiddle`
 - o `g++ -O -c -ocpxfiddle.o cpxfiddle.cc` (you can ignore warning)
 - o `g++ -O cpxfiddle.o -o cpxfiddle`
- Clean sources in `$HOME/SAR/EXEC/cpxfiddle.o`
- Store the installed source `cpxfiddle.cc` e.g. in `$HOME/SAR/EXEC/Sources_Installed`
- Update `$PATHTOCPXFIDDLE` in your `.bashrc`. It must be something like `export PATHTOCPXFIDDLE=${HOME}/SAR/EXEC/`

gnuplot:

- At Terminal:
 - o `sudo port install gnuplot` (Macport) or `brew install gnuplot`
 - o `which gnuplot`
- Ensure that the path provided by the former command displays the same path as `$PATHGNU` in your `.bashrc`. It must be something like `export PATHGNU=/opt/local/bin`

osascript:

- should be by default on your mac. Test at Terminal with : `osascript -h`

python:

- At Terminal:

Create a virtual environment. Python 3.11 is used, except on macOS 10.15 (Catalina) and older where no OpenCV wheel exists for 3.11 and python 3.10 must be used instead (replace 311 by 310 and 3.11 by 3.10 below):

- o `sudo mkdir -p /opt/local`
- o `sudo chown -R $USER:staff /opt/local`

with MacPorts:

- o `sudo port -f install python311 py311-pip`
- o `sudo port select --set python3 python311`
- o `sudo port select --set pip pip311`
- o `PYBIN=/opt/local/bin/python3.11`

or with Homebrew:

- o `brew install python@3.11`
- o `PYBIN="$(brew --prefix python@3.11)/bin/python3.11`

then create the environment. The `--system-site-packages` option is required so that the environment can see the modules installed system wide by MacPorts or Homebrew (typically the gdal bindings):

- `rm -rf /opt/local/amster_python_env`
- `$PYBIN -m venv /opt/local/amster_python_env --system-site-packages --upgrade-deps`
- `sudo chown -R $USER:staff /opt/local/amster_python_env`
- `/opt/local/amster_python_env/bin/pip install --upgrade pip setuptools wheel`

Load the necessary modules by creating a requirements.txt and installing it. The modules common to all the macOS versions are:

- `echo "numpy==1.26.4" > /opt/local/requirements.txt`
- `echo "scipy==1.11.4" >> /opt/local/requirements.txt`
- `echo "matplotlib==3.8.2" >> /opt/local/requirements.txt`
- `echo "shapely==2.0.2" >> /opt/local/requirements.txt`
- `echo "utm==0.7.0" >> /opt/local/requirements.txt`
- `echo "networkx==3.2.1" >> /opt/local/requirements.txt`

pyproj, PyQt6 and the gdal python bindings depend on the macOS version. On macOS 10.15 (Catalina) and older, proj, gdal and the gdal bindings come from MacPorts (replace py311 by py310 if you installed python 3.10):

- `sudo port install proj gdal py311-gdal`
- `export PROJ_DIR=/opt/local`
- `echo "pyproj==3.4.1" >> /opt/local/requirements.txt`
- `echo "PyQt6==6.5.3" >> /opt/local/requirements.txt`

On macOS 11 (Big Sur) and 12 (Monterey), numba and llvmlite must also be pinned to their last versions compatible with these OS:

- `echo "pyproj==3.6.1" >> /opt/local/requirements.txt`
- `echo "PyQt6==6.7.1" >> /opt/local/requirements.txt`
- `echo "numba<=0.62.1" >> /opt/local/requirements.txt`
- `echo "llvmlite<=0.45.1" >> /opt/local/requirements.txt`

On macOS 13 (Ventura) and newer, the latest pyproj is used:

- `echo "pyproj" >> /opt/local/requirements.txt`
- `echo "PyQt6==6.7.1" >> /opt/local/requirements.txt`

From macOS 11 on (Big Sur up to Tahoe with Homebrew), there is no MacPorts port nor Homebrew formula for the python gdal bindings (osgeo): they must be built with pip, matched to the version of the gdal really installed on your system:

- `/opt/local/amster_python_env/bin/pip install numpy wheel setuptools`
- `/opt/local/amster_python_env/bin/pip install "gdal[numpy]==$(gdal-config --version).*"`

OpenCV also depends on the python version installed above, that is 4.6.0.66 with python 3.10 (macOS 10.15 and older) or 4.9.0.80 with python 3.11:

- `echo "opencv-python==4.9.0.80" >> /opt/local/requirements.txt`
- `echo "opencv-contrib-python==4.9.0.80" >> /opt/local/requirements.txt`

And finally the remaining modules, common to all versions:

- `echo "geopandas==0.14.3" >> /opt/local/requirements.txt`
- `echo "scikit-gstat==1.0.18" >> /opt/local/requirements.txt`
- `echo "rasterio==1.3.9" >> /opt/local/requirements.txt`
- `echo "pandas==2.2.1" >> /opt/local/requirements.txt`
- `echo "glob2==0.7" >> /opt/local/requirements.txt`
- `echo "appscript==1.3.0" >> /opt/local/requirements.txt # Only for Mac`

Then install everything in the virtual environment:

- `/opt/local/amster_python_env/bin/pip install -r /opt/local/requirements.txt`

To ensure using the here installed Python3 version and modules in the virtual environment, all the python scripts must start with the line

```
#!/opt/local/amster_python_env/bin/python
```

Note: gdal itself is not installed with pip. The gdal used by the scripts is the one installed with MacPorts or Homebrew in section 1; only its python bindings are added as explained above. Installing a pip gdal on top of it leads to version conflicts.

3) Installing AMSTer software

AMSTer consists of several packages that need to be obtained or downloaded from different sources:

- **AMSTerEngine** (core parts: InSAR and MSBASTools):
 - See **Preliminary information/Sources and codes** above for Github instructions, or
 - contact: dderauw@ecgs.lu
 - **AMSTer Toolbox (SCRIPTS_MT: Main Toolbox mass processing scripts)**:
 - See **Preliminary information/Sources and codes** above for Github instructions, or
 - contact ndo@ecgs.lu
 - **MSBAS** (2D or 3D inversion time series):
 - See **Preliminary information/Sources and codes** above for Github instructions, or
 - Download from: <https://doi.org/10.4095/313749> or
 - Request code tuned for AMSTer with a make file adapted to Mac and Linux, contact ndo@ecgs.lu
- In case of problem, contact sergey.samsonov@canada.ca

All these components are gathered in the AMSTer_Distribution repository. The installer script offers to download (or update) it for you in `$HOME/SAR/EXEC/AMSTer_Distribution`, that is where it keeps all the sources it installs from. To do it manually, either clone it, which is recommended since it can then be updated later with a simple git pull:

- `mkdir -p $HOME/SAR/EXEC`
- `cd $HOME/SAR/EXEC`
- `git clone --depth 1 --branch main`
https://github.com/AMSTerUsers/AMSTer_Distribution.git

or download the zip archive of the branch and decompress it. Beware that the archive expands into a directory named AMSTer_Distribution-main, which must be renamed:

- `curl -fL -o AMSTer_Distribution-main.zip`
https://github.com/AMSTerUsers/AMSTer_Distribution/archive/refs/heads/main.zip
- `unzip -o AMSTer_Distribution-main.zip -x "__MACOSX/*"`
- `mv -f AMSTer_Distribution-main AMSTer_Distribution`

On a Mac, remember that `/usr/bin/git` is only an xcode-select shim (see section 1): if the Command Line Tools are not properly installed and active, use the git provided by Homebrew or MacPorts, or use the zip archive.

3.1 Compile **AMSTerEngine**:

If executable files were provided (zip file),

- simply unzip it and copy the binaries in `$HOME/SAR/AMSTer/AMSTerEngine`
- then store the zipped file in `$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older/Vmostrecent_AMSTerEngine` where *mostrecent* is the date of the last version, which is in the name of the zip file. This date must be as YYYYMMDD and will be used by some scripts to track (and log) the version used during processing.

- Update `$PATH` in your `.bashrc`. It must be something like `PATH=$PATH:$HOME/SAR/AMSTer/AMSTerEngine`

If you were provided with the source code, it comes as a tar.xz archive named AMSTerEngineYYYYMMDD.tar.xz, where YYYYYMMDD is the date of the version. That date is used by some scripts to track (and log) the version used during processing. Store the archive in

- `$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older/VYYYYMMDD_AMSTerEngine/`

Beware: the path to the sources must not contain any white space, otherwise the compilation fails. Then, in a Terminal, decompress the archive:

- `cd $HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older/VYYYYMMDD_AMSTerEngine`
- `tar -xf AMSTerEngineYYYYMMDD.tar.xz`

The recent distributions contain a second archive in a sub directory named Archives. If that directory exists, decompress that archive as well:

- `cd AMSTerEngineYYYYMMDD/Archives`
- `tar -xf Mas*.tar.xz`

Compile the InSAR part. `USEOPENMP=YES` enables the parallelisation and is recommended (it requires libomp, installed in section 2); without it, simply drop that option.

PKGMGR tells the makefile which package manager provides the compiler and the libraries, that is PKGMGR=port on macOS <26, and PKGMGR=brew from macOS 26 (Tahoe):

- `cd InSAR/sources`
- `make USEOPENMP=YES PKGMGR=port`

Copy the history file and move the binaries in `$HOME/SAR/AMSTer/AMSTerEngine`. The bin directory must contain `initInSAR`; if it does not, you are not in the right directory:

- `cp _History.txt $HOME/SAR/AMSTer/AMSTerEngine/`
- `mv -f ./bin/* $HOME/SAR/AMSTer/AMSTerEngine/`

Compile the MSBASTools the same way. Here the bin directory must contain `getLineThroughStack`:

- `cd ../../MSBASTools/sources`
- `make USEOPENMP=YES PKGMGR=port`
- `mv -f ./bin/* $HOME/SAR/AMSTer/AMSTerEngine/`

Clean (delete) the decompressed directories; keep only the tar.xz archive in `$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older/VYYYYMMDD_AMSTerEngine`.

Update `$PATH` in your `.bashrc`. It must be something like `PATH=$PATH:$HOME/SAR/AMSTer/AMSTerEngine`

Note that if you are provided with an updated version of AMSTer Engine (source code), you can simply update it by copying the zipped file it in `$HOME/SAR/EXEC` and run the script `/$HOME/SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT_Ndo/UpdateAMSTerEngine.sh` with the path to the zipped sources file as parameter.

3.2 Compile **MSBAS**:

- The MSBAS sources are provided as zip files in the AMSTer_Distribution, in MSBAS_sources/msbasvNN/. Several versions exist, e.g.
 - o msbas_20201009_wExtract_Unified_20220919_Optimized_v1.2.2_Gilles.zip for msbasv4 (2D) running in parallel on all the available cores and compatible with Homebrew,
 - o msbas_20201009_wExtract_Unified_20220919_Optmized_v1.3.2_Full3D.zip for the full 3D decomposition, or
 - o msbas_v10_20230601_Gilles.zip for msbasv10.

Copy the chosen one where you want to keep the installed sources, e.g. in `$HOME/SAR/EXEC/Sources_Installed/Former_msbas/`, then decompress it:

```
cd $HOME/SAR/EXEC/Sources_Installed/Former_msbas
unzip -o msbas_..._Optimized_vX.Y.Z_Name.zip -x "__MACOSX/"
```

- Enter the directory really created by unzip: it is not necessarily named after the zip file. A zip renamed e.g. msbas_..._V1_1.zip (dots turned into underscores by a mail server or a copy on a FAT disk) still expands into msbas_..._V1.1/. List it with:

```
ls -d msbas_*/
cd msbas_..._Optimized_vX.Y.Z_Name
```

- These archives provide several makefiles, some flavoured for MacPorts (they hard-code paths like `/opt/local/bin/clang++-mp-21`) and some for Homebrew. Never mix the two flavours.

Copy the one matching your package manager over Makefile, taking the one that exists in the list below (do not delete the current Makefile before having checked that its replacement is there, as some distributions provide only part of them):

- o Homebrew (macOS 26 Tahoe and later):
 - Makefile_brew_cpp17,
 - msbasv10_Makefile_brew
- o MacPorts on macOS 10.15 (Catalina) and older:
 - Makefile_clang14_cpp11,
 - Makefile_clang21_cpp17,
 - msbasv10_Makefile_port
- o MacPorts on macOS 11 (Big Sur) and newer:
 - Makefile_clang21_cpp17,
 - Makefile_clang14_cpp11,
 - msbasv10_Makefile_port

e.g.

```
cp -f Makefile_clang21_cpp17 Makefile
```

- Delete any msbas binary shipped within the zip or left by a former build before compiling: the msbasv4 distribution ships a prebuilt x86_64 binary and a failed compilation would leave it in place, where it could be taken for a successful build and installed for the wrong architecture:

```
rm -f msbasv4 msbasv4_3D msbasv10 msbasv10_3D msbas
```

- then compile and copy the executable really produced in `$HOME/SAR/AMSTer/MSBAS` (it seems that you can ignore warnings; most recent compilers are more sensitive):

```
make all
cp -f msbasvNN $HOME/SAR/AMSTer/MSBAS/
```

- msbas_extract is only provided with msbasv4 and msbasv4_3D (its sources are identical in both). With these versions, compile it as well, again with the Makefile matching your package manager (msbas_extract_Makefile_brew or msbas_extract_Makefile_port):

```
cd msbas_extract
cp -f msbas_extract_Makefile_port Makefile
rm -f msbas_extract
make
cp -f msbas_extract $HOME/SAR/AMSTer/MSBAS/
```
- If the compilation fails, check first the dependencies, which differ between the msbas generations: msbasv4 and v4_3D need clang (llvm with Homebrew) + gdal + libomp, while msbasv10 and v10_3D need mpich + gdal + libomp.
- Former versions of msbas already present in \$HOME/SAR/AMSTer/MSBAS should be kept as a backup in \$HOME/SAR/EXEC/Sources_Installed/Former_msbas/ before installing the new one, and the sources must be kept there as well (only remove the __MACOSX, .DS_Store and *.o leftovers).
- Update \$PATH in your .bashrc. It must be something like
PATH=\$PATH:\$HOME/SAR/AMSTer/MSBAS

Note:

The version [msbas_20201009_wExtract_Unified_20220919_Optimized_v1.3_Full3D.zip](#) is dedicated to very specific usage, that is when enough diversity in viewing geometries is achieved to perform full 3D inversion. Usually, this can only be achieved when data are acquired along Ascending and Descending orbits by sensors with right and left looking capabilities. See AMSTer manual.

3.3 Get the **AMSTer Toolbox scripts at the right place:**

- Ensure that the scripts (obtained from Github or from unzipped file) are stored at the right place:
\$HOME/SAR/AMSTer/SCRIPTS_MT
- Update \$PATH in your .bashrc. It must be something like
PATH=\$PATH:\$HOME/SAR/AMSTer/SCRIPTS_MT
PATH=\$PATH:\$HOME/SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT
PATH=\$PATH:\$HOME/SAR/AMSTer/SCRIPTS_MT/zz_Uutilities_MT_NdO
PATH=\$PATH:\$HOME/SAR/AMSTer/SCRIPTS_MT/_cron_scripts
PATH=\$PATH:\$HOME/SAR/AMSTer/SCRIPTS_MT/optimtoolbox
PATH=\$PATH:\$HOME/SAR/AMSTer/SCRIPTS_MT/diagtoolbox
PATH=\$PATH:\$HOME/SAR/AMSTer/SCRIPTS_MT/AMSTerOrganizer
- The order matters: \${PATHGNU} and /opt/local/bin must come **before** the default system directories, so that the gnu commands and the python of the virtual environment have priority over the Mac ones, while the SCRIPTS_MT, AMSTerEngine, MSBAS, EXEC and curl directories are appended after.
With Homebrew, its own bin directory (/opt/homebrew/bin or /usr/local/bin) must also be added before them, for every Homebrew tool that is not mirrored into /opt/local/bin (gdal, ffmpeg, ImageMagick, mpich, llvm, python, gsl...).
- If scripts were receiver from a zipped file, it can be stored in
\$HOME/SAR/AMSTer/EXEC/Sources_Installed

3.4 Define the environmental variables:

AMSTerEngine, MSBAS and AMSTer Toolbox requires environmental variables that will be called from the scripts. These are :

`PATH_SCRIPTS=${HOME}/SAR/AMSTer` : path to where is located the directory `SCRIPTS_MT` (i.e. the directory containing all the scripts)

`PATHGNU=/opt/local/bin` : path to where the `gnu utilities` are stored (better to have it in the `$PATH` as well)

`PATHFIJI=/Applications/Fiji.app/Contents/MacOS/` : path to where `Fiji` application is

`PATHCONV=/opt/local/bin` : path to where the `convert` command is (from `ImageMagick`)

`PATHTOCPXFIDDLE=${HOME}/SAR/EXEC/` : path to where the `cpxfiddle` command is (from TU Delft)

`JAVA_HOME` may also be required by some applications. Get its value with `java -XshowSettings:properties -version 2>&1 > /dev/null | grep 'java.home'` and export it in your `.bashrc` (the path must be quoted if it contains white spaces). With Homebrew, use `$(brew --prefix openjdk)/libexec/openjdk.jdk/Contents/Home` instead, since its openjdk is keg-only.

The path to some disks where data, intermediate results and final results will be stored must also be defined by state variable.

At least 5 variables **must be defined**, that is:

`PATH_1650`, `PATH_3600`, `PATH_3601`, `PATH_3602` and `PATH_DataSAR`.

More can be defined, e.g.: `PATH_1660`, `PATH_3610`, `PATH_3611`, `PATH_3612`, `PATH_SynoData` and `PATH_HOMEDATA`, which are optional and used e.g. to spread the computation over more disks (see e.g. `__SplitSession.sh`).

Names are inherited from architecture at ECGS and will be called throughout the processing. For security reason, it is a good practice to have them pointing to different physical drives, if possible, but this is not mandatory; they can also point toward directories instead of disks, e.g.:

<code>PATH_1650=/Volumes/hp-1650-Data_Share1</code>	(disk or dir where data or results will be stored)
<code>PATH_3600=/Volumes /hp-D3600-Data_Share1</code>	(disk or dir where data or results will be stored)
<code>PATH_3601=/Volumes /hp-D3601-Data_RAID6</code>	(disk or dir where data or results will be stored)
<code>PATH_3602=/Volumes /hp-D3602-Data_RAID5</code>	(disk or dir where data or results will be stored)
<code>PATH_DataSAR=/Volumes /DataSAR/</code>	(disk or dir where auxiliary data will be automatically stored such as orbits, geoid etc... in a dir named <code>SAR_AUX_FILES</code>)

Note also that more disks can be defined and used e.g. for sharing the computation through several disks when intensive computation is required.

When processing ENVISAT or Sentinel 1 data, AMSTerEngine will require orbital information. It will usually pick them for you and store them automatically in dedicated directories where it will get them when required. These directories **must be defined** by the following state variable names:

`S1_ORBITS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB`
`ENVISAT_PRECISES_ORBITS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/ENV_ORB`
 (the `vor_gdr_d` sub directory must exist inside that directory)

To create DEM corrected from geoid height, the following state variable must be defined:

`EARTH_GRAVITATIONAL_MODELS_DIR=$PATH_DataSAR/SAR_AUX_FILES/EGM`

A directory named `EGM96` must exist in that directory and it must contain the geoid file `WW15MGH.DAC`. It can be downloaded from (link checked 05 10 2022):

<https://web.archive.org/web/20130314064801/http://earth-info.nga.mil/GandG/wgs84/gravitymod/egm96/binary/WW15MGH.DAC>

To create DEM, `EXTERNAL_DEMS_DIR` state variable needs to be defined in your `.bashrc` (e.g.: `EXTERNAL_DEMS_DIR=/YourPath/DEMS/`).

When you create your DEM (e.g. using `getSRTMDEM` function since AMSTerEngine V20241010– see manual), it is strongly advised to move (and rename if needed) the created DEM to a directory from where you will call it in the `ParametersFile.txt` files. By default, the DEM will have the same name than the given kml.

You can also create automatic water body mask using `getSRTMDEM` function since AMSTerEngine V20241010 (see manual). For that, you will need to have a state variable `EXTERNAL_MASKS_DIR`.

If an `EXTERNAL_MASKS_DIR` environmental variable is defined in your `.bashrc` (e.g.: `EXTERNAL_MASKS_DIR=/YourPath/MASKS/`), the waterBody mask will be saved in that `EXTERNAL_MASKS_DIR`. In that case, it is also **strongly advised to move (and rename if needed)** the created mask to a directory from where you will call it in the `ParametersFile.txt` files.

Finally, the following directories must exist in `${PATH_DataSAR}`; create them with:

```
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/DEM
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/EGM/EGM96
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/ENV_ORB/vor_gdr_d
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/ERS
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB/AUX_POEORB
mkdir -p ${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB/AUX_RESORB
```

and downloads the geoid file `WW15MGH.DAC` with `wget` in

`SAR_AUX_FILES/EGM/EGM96/` (see section 5 and the link above).

Create an empty and properly protected `.netrc` in your home directory if you do not have one yet :

```
touch .netrc
chmod 0600 .netrc
```

because it is needed for the S1 orbits, the SRTM DEM creation, etc. See section 7 for its content.

Two commented lines are also added at the end of the `.bashrc`, to be uncommented and adapted if one day you need to limit the hardware usage while running `msbas` parallelised with OpenMP:

`#export OMP_NUM_THREADS=10,8,4` and `#export OMP_NUM_THREADS=4,3,2`

4) Directory structure for data, auxiliary data and results

In addition to the structure of the installation directories, it is important to store your data, auxiliary data (DEM, masks, kml, orbits, geoid, parameters files...), intermediate results and final results in a structure as described below.

AMSTer expects a directory structure similar as what is shown below. Most of the directories where the intermediate and final results are stored will be created by the scripts based on the information provided in the [LaunchMTparameters.txt](#) file.

For those which must be created by the user (in full or based on name provided in the [LaunchMT-parameters.txt](#) file), it is required to follow the following architecture and directories naming.

In *italic green* is the information that must be adapted to your satellite(s), orbital mode(s) and region(s). These directories can be located in the disk(s) you want.

Note that, for safety reasons, it is advised to store e.g. raw data and data in csl format (i.e. from AMSTerEngine) on different hard drives, or the mass processed results and the msbas results on different hard drives etc....

Names in **red** are **mandatory**.

Disk 1 (i.e. **PATH_1650**):

PATH_1650/kml	: where to store kml for cropping, or for coherence check
PATH_1650/Param_files	: where to store your LaunchMTparameters.txt file by <i>/SAT/TRK....</i>
PATH_1650/ SAR_CSL/SAT/TRK/NoCrop	: where data in csl format are stored (incl. crops)
PATH_1650/SAR_SM/AMPLITUDES/ <i>SAT/TRK/Region</i>	: where amplitudes gif will be computed
PATH_1650/ SAR_SM/MSBAS/Region	: where compatible pairs for msbas are computed
PATH_1650/ SAR_SM/RESAMPLED/SAT/TRK	: where data resampled on Global Primary are saved

Disk 2 (i.e. **PATH_3600**):

PATH_3600/**SAR_DATA/SAT/...** : where raw data are stored (safer to keep them on a different hard drive than 1650)

Disk 3 (i.e. **PATH_3601**):

PATH_3601/ SAR_MASSPROCESS/SAT/TRK/Crop	: where directories with each pair computation details and results are stored
PATH_3601/ SAR_MASSPROCESS/SAT/TRK/Crop/Geocoded	: where geocoded maps are stored in sub dir Amp, Coh, Defo etc...
PATH_3601/ SAR_MASSPROCESS/SAT/TRK/Crop/GeocodedRasters	: where rasters of geocoded maps (for quick look) are stored

Disk 4 (i.e. **PATH_3602**):

PATH_3602/**MSBAS_RESULTS/Region** : where msbas series are computed

Maybe in one of these disks, or somewhere else where enough storage room is available:

\$PATH_DataSAR/SAR_AUX_FILES/DEM	: where DEMs are stored
\$PATH_DataSAR/SAR_AUX_FILES/EGM96	: where geoid is stored
\$PATH_DataSAR/SAR_AUX_FILES/MASKS/SAT/TRK	: where coherence masks are computed
\$PATH_DataSAR/SAR_AUX_FILES/ORBITS/SAT	: where orbits are stored

Note that **\$PATH_DataSAR** is the state variable defined in .bashrc !!

5) Check installation

5.1 The directory structure

At the end, you should have AMSTer installed in a folder named “SAR” as shown in **Fig. 1**.

```

/$HOME/SAR/EXEC
|--Executables (cpxfiddle, snaphu...)
|--Sources_Installed

/$HOME/SAR/AMSTer
|--AMSTerEngine
|   |--_Sources_AE/Older
|   |   |
|   |   |--[several sources of old versions]
|   |   |--V20250808_AMSTerEngine
|   |--Executables...
|
|--DOC
|   |--AMSTerEngine
|   |--AMSTer_Toolbox
|   |--Biblio
|   |--LOGOS
|   |--MSBAS
|
|--MSBAS
|   |--3Dsbas_Archives
|   |   |--msbasv4
|   |--Executables...
|
|--SCRIPTS_MT
|   |--_cron_scripts
|   |--AMSTerOrganizer
|   |--diagtoolbox
|   |--optimtoolbox
|   |--TemplatesForPlots
|   |--TSCombiFiles
|   |--zz_Uilities_MT
|   |--zz_Uilities_MT_Ndo
|   |--A lot of scripts...

```

Figure 1 – Directory structure required for the proper usage of AMSTer.

Notes:

- 1) The folders `/$HOME/SAR/AMSTer/AMSTerEngine/_Sources_AE/Older` is used to store the archives and the most recent sources of both components of AMSTerEngine, that is `InSAR` and `MSBASTools`.

Their latest executable version, when compiled, must be moved to `/$HOME/SAR/AMSTer/AMSTerEngine/`.

Hence, when a new version of these packages is available, no need to change the paths of the environmental variables.

- 2) Similarly, the folders `/$HOME/SAR/AMSTer/MSBAS/_3Dsbas_Archives` is used to store the archives and the most recent sources of both components of MSBAS, that is `MSBAS_EXTRACT` and `MSBASi` (where *i* stands for the version). Their latest executable version, when compiled, must be moved to `/$HOME/SAR/AMSTer/MSBAS/`.

Hence, when a new version of these packages is available, no need to change the paths of the environmental variables.

- 3) You can use the script **Check_Installation.sh** to check your installation. See script and manual.

You should also have auxiliary files stored in a folder named “**SAR_AUX_FILES**” in **DataSAR** as shown in **Fig. 2**.

```
$PATH_DataSAR/SAR_AUX_FILES/
|--DEM
|
|--EGM96
|   |--WW15MGH.DAC
|
|--MASKS
|
|--ORBITS
|   |--ENV_ORB
|   |--S1_ORB
```

Figure 2 – Directory structure required for the storage of auxiliary files required by AMSTer Toolbox.

Notes:

- 1) The folders `/$PATH_DataSAR/SAR_AUX_FILES/DEM/` is used to store the DEMs. That is where you store and sort manually your DEMs according to your needs.

It is advised to use another path than the one defined in the state variable `$EXTERNAL_DEMS_DIR`. This last one is used e.g. to create automatically the DEM using the AMSTerEngine command **getSRTMDem** (see manual). However, for the automation with AMSTer Toolbox, to avoid possible confusion, we recommend moving the DEM created in `$EXTERNAL_DEMS_DIR` to `/$PATH_DataSAR/SAR_AUX_FILES/DEM/` and indicate that last full path in the `LaunchMTparameters.txt` file for each of your run. If you move the DEM, **remember to change the path** indicated in the `DEM.txt` file accordingly to the new place where the DEM is stored.

The only exception when you may need to have DEMs in `$EXTERNAL_DEMS_DIR` is if you intend to use AMSTerEngine without the AMSTer Toolbox scripts (e.g. using **LazInSAR** command). Indeed, in that case, unless you manually change the configuration text files, it will expect the DEM in that directory.

- 2) The folders `/$PATH_DataSAR/SAR_AUX_FILES/EGM` **must** contain the geoid file in `EGM96/WW15MGH.DAC`.
- 3) The folders `/$PATH_DataSAR/SAR_AUX_FILES/MASKS` is not mandatory, but it is a convenient place to store the mask you would build and used during your processing.

It is advised to use another path than the one defined in the state variable `$EXTERNAL_MASKS_DIR`. This last one is used e.g. to create automatically the water body mask when creating the DEM using the AMSTerEngine command **getSRTMDem** (see manual). However, for the automation with AMSTer Toolbox, to avoid possible confusion, we

recommend moving the masks created in `$EXTERNAL_MASKS_DIR` to `/$PATH_DataSAR/SAR_AUX_FILES/MASKS/` and indicate that last full path in the `LaunchMTparameters.txt` file for each of your run. If you move the mask, **remember to change the path** indicated in the `MaskWB.txt` file accordingly to the new place where the mask is stored.

- 4) The folders `/$PATH_DataSAR/SAR_AUX_FILES/ORBITS` **must** contain the orbits for ENVISAT (if required) or Sentinel 1 (if required). These directories must be associated to the state variable `$ENVISAT_PRECISES_ORBITS_DIR` and `$S1_ORBITS_DIR` respectively.
- 5) To automatically create DEM using the AMSTerEngine command **getSRTMDEM** (since AMSTer Engine from October 2024, though seems obsolete nowadays), you need to have the following lines in your `.nertc` (see paragraph 7 below to create a `netrc`):

```
machine e4ftl01.cr.usgs.gov login yourlogin password yourpwd
```

A similar line will be required for downloading Copernicus products from ESA.

5.2 The environmental variables in .bashrc

At the end, your `./bashrc` must look like:

```
#
# .bashrc
#

[...]

# User specific environment and startup programs
PATH=/opt/local/bin:$PATH
PATH="$PATH:/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin"

BASH_ENV=$HOME/.bashrc
USERNAME=""
LANG=C

# export EDITOR=/usr/bin/edit
export EDITOR=/usr/bin/nano
export PAGER=/usr/bin/less

export USERNAME LANG BASH_ENV PATH

export JAVA_HOME=/Library/Internet\ Plug-Ins/JavaAppletPlugin.plugin/Contents/Home

# AMSTer PATHS
#####

PATH=$PATH:/Users/YourAccount/SAR/EXEC
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/MSBAS
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/AMSTerEngine
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/SCRIPTS_MT
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/SCRIPTS_MT/zz_Uilities_MT_Ndo
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/SCRIPTS_MT/zz_Uilities_MT
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/SCRIPTS_MT/_cron_scripts
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/SCRIPTS_MT/optimtoolbox
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/SCRIPTS_MT/diagtoolbox
PATH=$PATH:/Users/YourAccount/SAR/AMSTer/SCRIPTS_MT/AMSTerOrganizer

# AMSTer VARIABLES
#####

export PATH_HOMEDATA=/Users/YourAccount/
export PATH_3602=/Volumes/hp-D3602-Data_RAID5/
export PATH_3601=/Volumes/hp-D3601-Data_RAID6/
export PATH_3600=/Volumes/hp-D3600-Data_Share1/
export PATH_1650=/Volumes/hp-1650-Data_Share1/
export PATH_DataSAR=/Volumes/DataSAR/

export EARTH_GRAVITATIONAL_MODELS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/EGM
export ENVISAT_PRECISES_ORBITS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/ENV_ORB
export S1_ORBITS_DIR=${PATH_DataSAR}/SAR_AUX_FILES/ORBITS/S1_ORB

export EXTERNAL_DEMS_DIR=/APath/WhereToCreate/DEMs
export EXTERNAL_MASKS_DIR=/APath/WhereToCreate/MASKs

export PATH_SCRIPTS=/Users/YourAccount/SAR/AMSTer
export PATHCONV=/opt/local/bin
export PATHFIJI=/Applications/Fiji.app/Contents/MacOS/
export PATHGNU=/opt/local/bin
export PATHTOCPXFIDDLE=/Users/YourAccount/SAR/EXEC/
export PATH=$PATH

#export OMP_NUM_THREADS=10,8,4
#export OMP_NUM_THREADS=4,3,2
```

6) Operating with cron jobs

If you want to operate **AMSTer** from cron jobs, pay attention to the following:

- It is advised to launch scripts by redirecting the messages to /dev/null to avoid filling up the mailbox by ending the command line with

```
> /dev/null 2>&1
```

- If bash is not your default shell (as for the recent Mac OS X) and you want to run scripts from cronjobs, call the scripts in your crontab by calling bash as follow:

```
hh mm * * * /bin/bash YourScript.sh > /dev/null 2>&1
```

Example of crontab:

Always end command line launched by cron with > /dev/null 2>&1 to mute output and avoid filling up the mailbox. It will also prevent wrong count of running processes if any...

TO GET THE SCRIPTS RUNNING USING CMD LINES AND SCRIPTS STORED IN PATH AVAILABLE IN BASHRC RATHER THAN IN COMMON

/user/local/bin FOR INSTANCE, DO NOT FORGET TO SOURCE THE BASHRC AT THE BEGINNING OF THE SCRIPT !!

MM HH DoMonth Month DoWeek(0=sunday)

```
## *****
## ***** CONGO VVP *****
## *****
```

Step 1: S1 & CSK

```
45 04 * * * /bin/bash /Users/doris/SAR/SCRIPTS_MT/_cron_scripts/VVP_S1_Step1_Read_SMCReg_Pairs.sh > /dev/null 2>&1
```

```
30 12 * * * /bin/bash /Users/doris/SAR/SCRIPTS_MT/_cron_scripts/VVP_CSK_Step1_Read_SMCReg_Pairs.sh > /dev/null 2>&1
```

Step 2: S1 & CSK

```
31 06 * * * /bin/bash /Users/doris/SAR/SCRIPTS_MT/_cron_scripts/VVP_S1_Step2_Read_MassProc.sh > /dev/null 2>&1
```

```
31 07 * * * /bin/bash /Users/doris/SAR/SCRIPTS_MT/_cron_scripts/VVP_CSK_Step2_MassProc.sh > /dev/null 2>&1
```

Step 3: S1

```
02 21 * * * /bin/bash /Users/doris/SAR/SCRIPTS_MT/_cron_scripts/VVP_S1_Step3_MSBAS.sh > /dev/null 2>&1
```

```
02 02 * * * /bin/bash /Users/doris/SAR/SCRIPTS_MT/_cron_scripts/VVP_CSK_Step3_MSBAS.sh > /dev/null 2>&1
```

Website Update (once a week)

```
00 06 * * 0 /bin/bash /Users/doris/SAR/SCRIPTS_MT/_cron_scripts/rsync_satellite_images_to_virunga_volcanoes.sh > /dev/null 2>&1
```


7) Downloading the Sentinel-1 orbits

Processing Sentinel-1 data requires orbits files, either the Near Real Time predicted (PRE) orbits available 180 minutes after the acquisition, or the Non Time Critical Precises (POE) orbits available typically 20 days after the acquisition.

These can be downloaded automatically using the AMSTer Engine function **updateS1Orbits** as a transparent step in the processing, either from ESA or from ASF (Alaska SAR Facility).

In both cases, it requires to have **at least AMSTerEngine V20231213** and a login and a password to be obtained from ESA or ASF, which must be stored in a file named **.netrc** in your home directory. That file must contain one line for either ESA and/or ASF login as follow:

```
machine identity.dataspace.copernicus.eu login <YourLogin> password <YourPassword>
machine urs.earthdata.nasa.gov login <YourLogin> password <YourPassword>
```

The first line is dedicated to the access the ESA server <https://dataspace.copernicus.eu> (this dataspace server replaces the <https://scihub.copernicus.eu> server that was closed in November 2023), while the second line is dedicated to the access the ASF server.

Of course, **<YourLogin>** and **<YourPassword>** must be replaced (without the **<** **>** symbols) with the login and password obtained from the provider:

- For ESA, visit <https://dataspace.copernicus.eu> and proceed to the registration by following the instructions provided on the web site;
- For ASF, visit urs.earthdata.nasa.gov and proceed to the registration by following the instructions provided on the web site. When registered and logged in, you must also:
 - click on “My Profile”, then menu “Applications > Authorized Apps”.
 - At the bottom, click on button “APPROVE MORE APPLICATIONS”
 - In the “Search” bar, type “Sentinel”
 - Click on “AUTHORIZE” button next to each application named with Sentinel 1.

Then in a terminal on your computer type:

```
> cd ~
> touch .netrc
> echo "machine urs.earthdata.nasa.gov login <YourLogin> password <YourPassword>" > .netrc
> chmod 0600 .netrc
```

Then

```
> cd ~
> touch .urs_cookies
```

For more information, see: <https://wiki.earthdata.nasa.gov/display/EL/How+To+Access+Data+With+cURL+And+Wget>.

With such a **.netrc** file, providing that your destination directory **S1_ORBITS_DIR** is set up and defined as state variable in your **.bashrc**, you can download the orbits from ESA or ASF by launching either **updateS1Orbits** or **updateS1Orbits -ASF** respectively. This is usually done from the scripts and you do not have to worry about it.

8) Updates in this manual

V 4.1 (October 26 2022):

- add installation of gnu parallel
- add chapter “7) Updates in this manual”
- correct typo in Version number in header

V 4.3 (April 20 2023):

- Refer to last versions of software and scripts
- Check against installer script

V 5.0 (Aug 30 2023):

- Major changes in dirs and files naming
- Take into account new AMSTer Engine V20230828 which is mandatory

V 6.0 (Oct 25 2023):

- Major changes related to the renaming of MasTer as AMSTer
- Information about public GitHub access to source code and licensing
- Typos and varia

V 6.0.1 (Nov 14 2023):

- Update Macport jdk19 as jdk20

V 6.1 (Dec 15 2023):

- Data server change at ESA in Nov. 2023 requires new .netrc file for allowing the download of Sentinel-1 orbits.

V 6.1.1 (Jan 03 2024):

- Update msbas zip source code V1.2 to avoid compilation error with most recent Mac OSX version

V 6.2.1 (Oct 10 2024):

- Update some python instructions
- Add DEMS and MASKS state variables to cope with new function getSRTMDDEM from AMSTerEngine from October 2024
- Add info about .netrc for SRTM download

V 6.2.2 (Nov 13 2024):

- Information about
msbas_20201009_wExtract_Unified_20220919_Optimized_v1.3_Full3D.zip
- Cosmetic

V 6.2.3 (Aug 07 2025):

- New diagnostic toolbox (by Delphine Smittarello)
- Python packages scikit-gstat and geopandas
- Info about python 3 system-wide installation
- Clang version 20 is mandatory from AMSTerEngine_20250805

V 6.3.0 (Aug 14 2025):

- Info about python 3 installed in virtual environment

V 6.3.1 (Feb 10 2026):

- Correct path to installer when downloaded from Github Distribution repository
-

V 7.0 (Aug 20 2026):

- Update license information
- Refer to manual for installation using the installer script
- Update to distribution release from August 20, 2026:
 - o Add Xcode
 - o Use Homebrew for OSX from version 26.6 (Tahoe)
 - o Manual installation procedure checked line by line against AMSTer_install.sh V7.0 (Aug 06 2026) and updated accordingly with AI assistance.